

Autonomous Architectures for High-Frequency Sports Betting Exchanges: A Definitive Guide to Agentic System Generation

The rapid evolution of artificial intelligence has precipitated a profound paradigm shift in software engineering, transitioning the discipline from manual syntax generation to intent-driven, autonomous agentic workflows. This transition, often colloquially referred to as "vibe coding" or agent-first development, empowers domain experts to orchestrate complex software builds through high-level natural language directives, drastically altering the software development lifecycle.¹ Coined in early 2025 by AI researchers, the concept of vibe coding describes a workflow where the primary engineering role shifts from writing code line-by-line to guiding an AI assistant that autonomously generates, refines, and debugs an application.¹ Industry forecasts indicate that by the end of 2026, approximately sixty percent of all new software code will be AI-generated, representing an acceleration that redefines enterprise architecture.³

However, constructing enterprise-grade, high-frequency trading platforms—such as a peer-to-peer Sports Betting Exchange—requires an unprecedented level of architectural rigor that transcends basic exploratory vibe coding. Such systems demand ultra-low latency matching engines, complex financial liability calculations, deterministic state management, and distributed microservices capable of handling massive concurrency during live sporting events.⁴ "Pure" vibe coding, which relies entirely on trusting an AI's unverified output for rapid prototyping, is wholly insufficient for financial applications where race conditions or latency spikes can result in catastrophic monetary losses.¹ Instead, developers must employ "responsible AI-assisted development," utilizing agentic development platforms like Google Antigravity or Gemini CLI to orchestrate parallel agent teams operating under strict architectural constraints.¹

To successfully deploy a betting exchange utilizing autonomous AI agents, the prompt engineering must transcend a simple list of features. It requires a "master-level" meta-prompt: an exhaustive, self-contained constitution that dictates the technology stack, orchestrates parallel agent personas, establishes self-healing debugging loops, manages context saturation, and automates deployment through Model Context Protocol integrations.⁹ This comprehensive report delineates the underlying technologies required for a high-performance sports betting exchange, the theoretical frameworks governing autonomous agentic systems, and the definitive master prompt designed for execution within advanced environments to autonomously build the platform from inception to production.

Financial Architecture of a Peer-to-Peer Betting

Exchange

Before an autonomous agent can construct an exchange, the fundamental financial and architectural constraints must be explicitly defined. The traditional sportsbook model and the peer-to-peer betting exchange model operate on entirely different mathematical paradigms. A traditional sportsbook operates by setting fixed odds and accepting bets directly against the house, embedding a profit margin—known as the vigorish or juice—into the odds themselves.¹² The traditional bookmaker carries the full risk of the event and must constantly adjust odds internally to balance their books and protect their position.¹²

In contrast, a sports betting exchange operates as a neutral peer-to-peer marketplace where users bet exclusively against one another, eliminating the house's exposure to event outcomes.¹² The exchange acts solely as a facilitator, connecting the two sides of a wager and extracting a predefined commission, typically ranging from two to five percent, exclusively from the net winnings of the victorious party.¹³ This model aligns the platform's success with the trading volume rather than the sporting outcome.

Feature	Traditional Fixed-Odds Sportsbook	Peer-to-Peer Betting Exchange
Odds Determination	Set internally by the bookmaker's algorithms.	Determined entirely by user market activity and liquidity.
Profit Mechanism	Margin (vigorish) baked into the odds structure.	Commission (2-5%) charged on net winnings.
Risk Bearer	The bookmaker carries full financial exposure.	Risk is distributed entirely among the participating bettors.
Betting Flexibility	Users can only place bets on outcomes offered by the house.	Users can place "back" bets (predicting a win) or "lay" bets (predicting a loss).
Transparency	Closed system; internal risk	Open order book; market depth

	exposure drives odds changes.	and trends are visible to all users.
--	-------------------------------	--------------------------------------

The Mathematics of Back and Lay Betting

The most critical logic the autonomous agent must implement involves the distinction between "back" and "lay" betting, particularly concerning liability calculation. When a user places a back bet, they are predicting that a specific outcome will occur.¹³ The financial mathematics for a back bet are straightforward, mirroring traditional betting where the maximum potential loss is strictly limited to the initial stake wagered.¹⁵

Conversely, laying a bet means acting as the bookmaker and predicting that an outcome will not occur.¹² When a user lays a bet, they are accepting another user's back bet at agreed-upon odds. Consequently, the layer's liability—the amount they stand to lose if the event does occur—can exponentially exceed their initial stake.¹⁵ The autonomous agent must be programmed to calculate and enforce this liability with absolute precision to prevent system insolvency.

The mathematical formulation for calculating lay liability is expressed as:

$$Liability = (Stake \times DecimalOdds) - Stake$$

To illustrate the stark contrast in risk exposure, consider the implementation of these calculations at varying decimal odds.

Bet Type	Target Outcome	Decimal Odds	Stake Entered	Potential Profit	Liability (Maximum Loss)
Back	Team A Wins	6.00	\$10.00	\$50.00	\$10.00
Lay	Team A Does Not Win	6.00	\$10.00	\$10.00	\$50.00
Back	Team B Wins	41.00	\$10.00	\$400.00	\$10.00

Lay	Team B Does Not Win	41.00	\$10.00	\$10.00	\$400.00
-----	---------------------	-------	---------	---------	----------

The order matching engine must continuously enforce stringent risk management protocols, ensuring that no user can place a lay bet unless their digital wallet contains sufficient available funds to cover the maximum calculated liability.¹⁵ When a back bet and a lay bet intersect at agreed-upon odds within the order book, the engine matches them, immediately locking the backer's stake and the layer's liability in a secure escrow ledger until the sporting event concludes and settlement occurs.¹⁷

Furthermore, the autonomous agent must implement market-making algorithms designed to shape the order book. These algorithms are responsible for moving lines in response to betting pressure, external signals, and dynamic limits, utilizing correlation matrices and scenario analysis to assess how specific match narratives affect the book's overall liquidity and exposure.¹⁸ Risk clustering must be implemented to group exposure by teams, players, and game scripts, enforcing risk caps at the cluster level to manage how losses could theoretically accumulate during high-impact events.¹⁸

The LMAX Disruptor and Low-Latency Matching Engines

The central nervous system of any financial exchange is the order matching engine, which must process market and limit orders instantly as odds fluctuate rapidly during live, in-play events.¹² Traditional software architectures typically rely on multithreaded implementations utilizing standard blocking queues to pass data between processing stages. However, extensive performance benchmarking reveals that the latency costs associated with standard queues are dramatically slow, often in the same order of magnitude as disk input/output operations.²⁰ The conflation of concerns in conventional approaches leads to severe thread contention, context-switching overhead, and kernel-level locking, which can add hundreds of microseconds to the overall latency footprint.²⁰

To achieve the sub-microsecond latency required for high-frequency trading, modern exchanges utilize the LMAX Disruptor pattern.⁶ The Disruptor is an inter-process concurrency and messaging pattern designed to act as a highly efficient, lock-free messaging infrastructure between producing and consuming threads.²¹

The LMAX Disruptor architecture relies on several fundamental computer science principles that the autonomous AI agent must be instructed to implement flawlessly.

Disruptor Component	Architectural Function	Performance Benefit
Ring Buffer	A pre-allocated, contiguous array in memory used for queueing events.	Eliminates dynamic memory allocation during runtime, completely bypassing garbage collection pauses.
Mechanical Sympathy	Designing data structures that align with modern CPU architecture cache lines.	Utilizes cache-line padding to prevent false sharing, ensuring threads do not invalidate each other's CPU caches.
Lock-Free Concurrency	Utilizing memory barriers and atomic operations rather than mutual exclusion (mutex) locks.	Drastically reduces latency by avoiding kernel-level thread suspension and signaling overhead.
Batch Consumption	Allowing consumers to process multiple available events in a single operation.	Highly efficient for downsampling snapshot-based events or updating order book levels during traffic spikes.

Testing has demonstrated that the mean latency using the Disruptor pattern for a three-stage pipeline is three orders of magnitude lower than an equivalent queue-based approach, processing upwards of six million transactions per second with a mean latency of fifty-two nanoseconds, compared to tens of thousands of nanoseconds for traditional array blocking queues.²⁰

Implementing the Order Book in Rust

While the LMAX Disruptor was originally conceived to bypass the limitations of the Java Virtual Machine—specifically the inability to create stack-allocated primitives and the unpredictable latency spikes caused by garbage collection—implementing the pattern in a modern systems programming language yields unparalleled performance.²⁴ The AI agent must be directed to construct the order book matching engine in Rust, a language that provides strict memory safety

guarantees without relying on a garbage collector.²⁴

A high-performance Rust order book utilizes atomics and lock-free data structures to maintain strict price-time priority for millions of concurrent orders.²⁶ The fundamental data structures must be carefully optimized. For instance, the system must utilize unsigned 64-bit integers (u64) for price and quantity representations to avoid the processing overhead and inaccuracies associated with floating-point arithmetic or negative values.²⁷ Epoch timestamps must be utilized to ensure strict first-in, first-out (FIFO) matching within identical price levels.²⁷

The Rust implementation must support various order types, including standard limit orders, market-to-limit orders, iceberg orders, and immediate-or-cancel (IOC) directives.²⁶ Furthermore, the architecture must support thread-safe price levels, allowing each price level within the order book to be independently and concurrently modified by multiple threads without blocking the entire pipeline.²⁶ Profiling and continuous benchmarking, such as flamegraphs and compiler hints for branch prediction, must be integrated into the agent's development loop to maintain ultra-fast execution speeds.²⁸

Data Persistence and Event-Driven Topologies

While the order matching engine relies entirely on in-memory Rust data structures for absolute speed, the broader betting exchange platform requires a robust, ACID-compliant relational database for long-term persistence, financial auditing, and user state management. PostgreSQL is the established industry standard for this persistence layer, supported by various high-performance ORMs and connection pooling strategies.⁵

The AI agent must design a highly normalized relational schema capable of managing the complex realities of sports tournaments. The core tables include tournaments, matches, teams, match statistics, and ledger balances.²⁹

Database Table	Primary Purpose	Architectural Considerations
tournament	Stores metadata regarding the overarching competition.	Must utilize JSON or JSONB columns for flexible fields like location coordinates or weather conditions, preventing the need for excessive table joins for non-relational data.
match	Acts as the centerpiece of the	Contains timestamps and status enums (e.g., Scheduled,

	event schedule.	In-Play, Completed). Must be heavily indexed on tournament_id for rapid filtering.
wallet_ledger	Immutable, append-only tracking of user balances.	Must enforce strict constraints and transactional integrity. Balances are derived by summing the ledger entries rather than updating a single mutable row, preventing race conditions.
order_history	Persists matched trades and canceled orders.	Crucial for regulatory compliance and providing historical datasets for training market-making algorithms and predictive models.

To bridge the gap between the ultra-fast Rust matching engine and the stateful PostgreSQL database, the macro-architecture must be entirely event-driven. The system requires distributed streaming platforms, such as Apache Kafka or Amazon Managed Streaming for Apache Kafka (MSK), to decouple the services.⁵ When the Rust engine executes a match, it publishes an event to a Kafka topic. Node.js backend microservices independently consume these events, asynchronously updating the PostgreSQL ledger and pushing real-time odds fluctuations to the Next.js frontend via WebSocket connections.⁵ Caching layers utilizing Redis are essential for serving immediate read requests regarding user balances and active order statuses, dramatically reducing the query load on the primary relational database.⁵

Principles of Autonomous Agentic Engineering

Deploying an architecture of this magnitude and complexity autonomously requires significantly more than a simple, conversational text prompt. It necessitates the initialization of a sophisticated multi-agent system operating within an agent-first development platform.⁷ The master prompt must configure the environment to manage complex reasoning loops, mitigate memory degradation, and execute code safely within isolated sandboxes.

The Agent Loop and Copilot-Style Architectures

The core of any autonomous AI software engineering system is the iterative execution cycle known as the AI agent loop.³² At each iteration, the agent assembles context from available inputs, invokes a large language model to reason about the problem, selects a specific action or tool, executes that action, observes the resulting output, and feeds that observation back into the context for the next iteration.³² This process, closely modeled on the ReAct (Reason, Act, Observe) framework, allows models to perform significantly better than single-pass generation systems, as they interleave reasoning traces with actionable environment interactions.³²

Modern agentic IDEs, such as Google Antigravity, elevate this concept by introducing an Agent Manager surface.⁷ Unlike traditional linear chat interfaces found in standard coding assistants, the Agent Manager acts as a mission control center, allowing developers to spin up multiple agent threads that operate simultaneously across different workspaces.⁸ This enables true parallel orchestration, where one agent persona refactors a legacy frontend component while another independent agent simultaneously writes unit tests or deploys database migrations.⁸

The master prompt must explicitly mandate the creation of specific, parallel agent personas to divide the labor effectively.

Agent Persona	Defined Responsibilities	Core Tool Utilization
Lead System Architect	Oversees the overarching project state, reviews API contracts, and orchestrates phase gating between the other agents.	File system reading, Markdown artifact generation, and cross-agent communication protocols.
Rust Systems Engineer	Implements the LMAX Disruptor, lock-free order book data structures, and Foreign Function Interface (FFI) bindings.	cargo build tools, rustc compiler, and advanced memory profiling utilities.
Full-Stack Developer	Implements the Next.js frontend interfaces, PostgreSQL relational schemas, and real-time WebSocket communication	npm package manager, PostgreSQL clients, and browser-actuation tools for UI testing.

	layers.	
QA and Security Specialist	Generates comprehensive unit tests, strictly validates liability mathematics, and executes security vulnerability analysis.	Testing frameworks (pytest, jest) and MCP-integrated security scanning extensions.

Context Management and the Prevention of Context Rot

A primary failure mode observed in long-running autonomous development tasks is the phenomenon of context bloat and context rot.³⁵ When developers attempt to stuff a single, monolithic prompt with exhaustive documentation, coding standards, and architectural blueprints, the large language model suffers from context bloat.³⁵ The massive volume of information acts as a distractor, diluting the model's attention and causing it to suffer from the "lost in the middle" effect, where it ignores crucial instructions buried deep within the text.³⁵

Furthermore, as the execution loop continues over hours of development, the conversation history expands exponentially, leading to context rot. The model's ability to process the information reliably degrades; it begins forgetting earlier constraints, hallucinating variable names, and deviating from the initial architectural design.³⁵

To effectively combat context degradation, the autonomous prompt must enforce a strict Artifact-First Workflow, completely separating prompt engineering from application code.³⁶ Instead of relying on the ephemeral memory of the chat session, the agent must be instructed to document its plans, database schemas, and current state in persistent, lightweight Markdown files.³⁶

This is achieved through a rigid file structure that acts as the cognitive architecture for the AI. The project root must contain a .agents/ or .antigravity/ directory that stores these critical context files.³⁸

Artifact File	Architectural Purpose	Execution Mechanism
AGENTS.md	The authoritative behavioral rulebook for the entire project.	Acts as the single source of truth for all coding standards, agent personas, and systemic constraints. ³⁶

mission.md	Defines the high-level business goals and requirements.	Provides persistent context regarding the specific domain (e.g., the peer-to-peer betting exchange model). ³⁷
.cursorrules / .windsurfrules	IDE-specific configuration files.	Act as thin bootstraps that execute upon launch, forcefully redirecting the local AI agent to read the central AGENTS.md rulebook before taking any coding action. ³⁷
artifacts/plan_*.md	Deterministic planning documents.	Forces the agent into chain-of-thought reasoning, outlining the steps, API routes, and schema changes before writing a single line of executable code. ³⁷

Progressive Disclosure via Agent Skills

To further optimize token consumption and maintain razor-sharp focus during complex tasks, the prompt must utilize the concept of Agent Skills. Skills represent an open standard for extending an agent's capabilities through progressive disclosure.³⁹ Rather than loading all technical nuances into the primary context window, specialized knowledge is compartmentalized into individual directories, such as `.agents/skills/<skill-folder>/SKILL.md`.³⁹

Each SKILL.md file contains YAML frontmatter defining its name and a concise description of its utility.³⁹ When the agent evaluates a task, it scans the descriptions of available skills. If a skill appears highly relevant—for instance, a skill specifically documenting the algorithmic nuances of calculating lay liabilities—the agent dynamically reads the full contents of that specific SKILL.md file, applying the knowledge strictly to the task at hand.³⁹ This mechanism ensures that the AI is not burdened with PostgreSQL indexing strategies while it is attempting to write Next.js frontend components, drastically improving response accuracy and reducing API costs.³⁹

Self-Healing Execution Loops and MCP Deployment Automation

The true power of an autonomous developer agent lies not merely in its ability to generate

syntax, but in its capacity to compile, execute, test, and repair the software it creates.⁴³ This paradigm shift moves the system from generative AI to authentic agentic engineering.⁴⁴ The master prompt must embed a rigorous self-healing code-test-debug loop that allows the agent to iteratively refine the application without human intervention.⁴³

The self-healing architecture follows a strict, four-stage cyclical process.⁴⁴ First, in the Generate phase, the agent writes the initial code block based on the specifications outlined in the planning artifacts.⁴⁴ Second, in the Execute phase, the system utilizes terminal tools to automatically run the code within a local sandboxed environment, executing commands such as `cargo test` or `npm run dev`.⁹ Third, in the Analyze phase, if the execution fails, the agent captures the standard output, standard error logs, and stack traces directly from the shell.⁴⁴ Finally, in the Repair phase, the captured error logs are fed back into the agent's reasoning engine, allowing it to formulate a hypothesis, rewrite the faulty code, and restart the cycle.⁴⁴

However, unconstrained self-healing loops pose significant systemic risks. If the AI becomes trapped in a logic failure, it may enter an infinite loop, rapidly burning through API quotas while repeatedly applying the same incorrect fix.⁴⁴ The autonomous prompt must mitigate this risk by explicitly enforcing bounded retries.⁴⁶ The agent must be instructed to limit its repair attempts for any single issue—for example, a maximum of five iterations—incorporating exponential backoff and jitter to prevent thundering herd problems during network requests.⁴⁶ If the code fails to compile or pass tests after the retry limit is exhausted, the agent is mandated to log the failure in a dedicated artifact, halt the loop, and either request human intervention or pivot to an entirely different architectural approach.⁴⁶

Model Context Protocol and Command-Line Interfaces

To execute these self-healing loops and interact with the host environment, the agentic system relies heavily on the Model Context Protocol (MCP). MCP provides a standardized architecture for independent server processes to expose custom tools, context, and data to the language model safely and reliably.¹¹ Through MCP integrations, the agent gains the ability to securely read and write files, execute shell commands, and interact with external APIs directly from the terminal.⁴⁸

When utilizing tools like the Gemini CLI, the agent's capabilities and boundaries are defined by a `gemini-extension.json` manifest file.⁵⁰ This file dictates the specific MCP servers to load on startup and explicitly outlines security exclusions.⁴⁹ Security is paramount when granting an AI agent shell access; therefore, the configuration must follow the principle of least privilege.⁵⁰ The manifest allows developers to explicitly block dangerous commands—such as `run_shell_command(rm -rf *)`—ensuring that the CLI intercepts and denies destructive actions even if the model attempts to execute them during an erratic self-healing loop.⁵⁰

Furthermore, the development workflow can be significantly accelerated through the use of custom slash commands defined in local `.toml` files.⁴⁸ These commands act as reusable macro triggers that streamline complex interactions.⁴⁸ For instance, a `/plan` command can instruct the

agent to exclusively generate planning artifacts without touching source code, while a `/security:analyze` command can trigger an MCP server to scan the codebase for hardcoded secrets or SQL injection vulnerabilities, adhering to OWASP guidelines.⁴⁸

For the final stages of the project, a `/deploy custom` command can automate the entire continuous integration and deployment pipeline.⁵³ The agent can be instructed to generate multi-stage Dockerfile configurations, utilize MCP tools to interact with the Docker daemon, and execute deployment scripts to push the containerized application directly to cloud infrastructure, such as Google Cloud Run or a Virtual Private Server.¹¹

The Master God-Level Autonomous Meta-Prompt

The following section contains the exhaustive, highly structured prompt engineered for direct insertion into advanced agent-first development environments such as Google Antigravity IDE, Gemini CLI, Cursor, or Cline.⁷ This meta-prompt synthesizes all aforementioned architectural requirements, context management strategies, and self-healing protocols into a single, executable directive. It utilizes structured XML tagging for precise semantic parsing, delineating roles, architectural constraints, and execution loops to ensure the model comprehends the overarching systemic boundaries.⁵⁷

XML

`<meta_directive>`

You are an ultra-competent, autonomous multi-agent engineering swarm acting collectively as the Lead Architect, Rust Systems Engineer, Node.js Backend Developer, and QA Specialist. Your overarching mission is to autonomously build, debug, thoroughly test, and deploy a production-ready, ultra-low latency Sports Betting Exchange (SBE).

You will operate with complete autonomy, utilizing all available Model Context Protocol (MCP) tools, shell execution environments, and file system operations. You will not ask for permission to execute standard software development tasks; you will proactively plan, execute, observe logs, and self-heal.

`</meta_directive>`

`<core_architecture_spec>`

The target system is a high-frequency peer-to-peer Sports Betting Exchange. It facilitates a neutral marketplace connecting "Backers" (predicting an outcome will occur) and "Layers" (predicting an outcome will not occur).

1. Order Matching Engine (The Core):

- Language: Must be written in highly optimized Rust to eliminate garbage collection latency.

- Architecture: Must implement the LMAX Disruptor pattern. Utilize lock-free ring buffers, atomic operations, and explicit cache-line padding (mechanical sympathy) to prevent false sharing and achieve sub-50 microsecond latency.
- Logic: Must support strict Price-Time priority matching for both market and limit orders.
- Financial Mathematics: Must calculate and enforce complex financial liabilities flawlessly. The formula for lay bets is: $\text{Lay Liability} = (\text{Stake} * \text{Decimal Odds}) - \text{Stake}$. Escrow locking mechanisms must be integrated directly into the matching logic to prevent insolvency.

2. Backend Services & Persistence Topology:

- Language: Must be written in Node.js (TypeScript) utilizing the Express or Fastify frameworks.
- Integration: Must interface with the isolated Rust matching engine via Foreign Function Interface (FFI) or ultra-fast local Inter-Process Communication (IPC).
- Database: PostgreSQL must be used for relational state management. The schema must handle Users, Wallets, Tournaments (utilizing JSONB for variable metadata), Matches, and an Immutable Append-Only Ledger for financial transactions. Utilize Drizzle ORM or Prisma for type-safe queries.
- Event Distribution: Utilize Redis for state caching and Apache Kafka (or Redis Pub/Sub) for distributing real-time odds updates horizontally.

3. Frontend Interface:

- Framework: Next.js 15 (App Router), React 19, Tailwind CSS v4, and Shadcn UI.
- Connectivity: Implement robust WebSockets for real-time order book visualization, instantaneous odds fluctuation rendering, and immediate trade execution feedback.

</core_architecture_spec>

<autonomous_execution_framework>

To prevent context rot, infinite regression loops, and architectural drift over long-running sessions, you must strictly adhere to the following operational phases and self-healing protocols. You must not skip any phase.

Phase 1: Workspace Initialization & Context Grounding

Before writing any application source code, you must establish the cognitive architecture of the workspace to ensure persistent memory.

1. Create a root directory named ``.agents/``.
2. Generate ``.agents/AGENTS.md``. This is the supreme behavioral rulebook. Document the multi-agent personas, the technology stack, and the strict coding standards (e.g., "All Rust structs must derive ``Debug``, ``Clone``; All TypeScript must utilize strict typing; No ``any`` types permitted").
3. Generate ``.mission.md`` in the root directory detailing the business logic and financial goals of the peer-to-peer Sports Betting Exchange.
4. Generate ``.cursorrules`` or ``.antigravity/rules.md`` containing a thin bootstrap directive that forces any local AI agent to read ``.AGENTS.md`` and ``.mission.md`` before taking any action.

5. Generate a `GEMINI.md` file (if operating in Gemini CLI) defining the high-level intent, minimal permissions, and explicit MCP tool usage guidelines.

Phase 2: Architectural Artifact Generation and Phase Gating

1. Create a directory named `artifacts/`.
2. Generate `artifacts/plan\_001.md`. This file must contain the sequential execution plan, complete database schema definitions, RESTful API contracts, and WebSocket event payloads.
3. DO NOT proceed to Phase 3 until `artifacts/plan\_001.md` is exhaustively populated, logically sound, and verified against the core architecture spec.

Phase 3: Implementation & Parallel Execution

Execute the detailed development plan. If you have access to an Agent Manager (e.g., Antigravity IDE), simulate parallel execution by strictly modularizing tasks.

- Step A: Build and migrate the PostgreSQL schema.
- Step B: Build the Rust order book and LMAX Disruptor structures. Implement rigorous, deterministic unit tests for the Liability calculations.
- Step C: Build the Node.js API layer and event publishers.
- Step D: Build the Next.js frontend components.

Phase 4: The Self-Healing Code-Test-Debug Loop

You must never leave broken, uncompileable, or failing code. For every component you build, you must enter a strict self-healing loop:

1. **Execute**: Run the compiler or test suite utilizing shell commands (e.g., `cargo test`, `npm run build`, `npx tsc`).
2. **Observe**: Capture the standard output (stdout) and standard error (stderr) logs via your shell execution tool.
3. **Analyze**: If an error occurs (e.g., Rust borrow checker violation, Node.js module resolution failure, PostgreSQL connection timeout), read the stack trace meticulously.
4. **Repair**: Formulate a technical hypothesis, apply the fix to the specific file, and loop back to step 1.
5. **Bounded Retries**: You are strictly limited to a maximum of 5 repair attempts per isolated issue. If a failure persists beyond 5 attempts, you must document the blocker in `artifacts/error\_log.md`, halt the loop, and pivot to an alternative architectural solution or mock the failing component to proceed.

Phase 5: Automated Deployment Integration

Once all test suites pass deterministically, utilize MCP shell tools to containerize the architecture.

1. Generate an optimized, multi-stage `Dockerfile` for the Node.js backend and the Next.js frontend, and a highly optimized release build for the Rust engine.
2. Generate a comprehensive `docker-compose.yml` to orchestrate the services, including PostgreSQL and Redis containers.
3. Execute local deployment commands (e.g., `docker compose up -d --build`) to verify

environment integrity.

</autonomous_execution_framework>

<skill_injection_directives>

To optimize token usage and prevent context saturation, implement specific Agent Skills using the principle of progressive disclosure.

1. Create `.agents/skills/rust-orderbook/SKILL.md`. Document the exact arithmetic formulas for limit orders, market orders, matching priority, and liability validation algorithms.
2. Create `.agents/skills/db-schema/SKILL.md`. Document the PostgreSQL indexing strategies required for high-frequency ledger queries and ACID compliance.

Whenever you encounter subsequent tasks related to these specific domains, you must read the respective `SKILL.md` file to refresh your specialized context before generating code.

</skill_injection_directives>

<critical_constraints_and_security>

- SECURITY FIRST: Never hardcode API keys, database credentials, or secret tokens. Always utilize environment variables (`.env`) and inject them securely into the application context.
- DETERMINISM: Ensure all financial calculations use precise numeric types. Utilize `Decimal` types in PostgreSQL and `u64` integers representing the smallest currency denomination (e.g., cents) in Rust to completely avoid floating-point inaccuracies.
- NO LAZY CODE: Never use placeholders like `// implementation goes here`, `// TODO`, or `...rest of code`. You must write complete, production-ready, vertically integrated, copy-pasteable files.
- AUTONOMY: You are an autonomous agent. Read this prompt, internalize the directives, and begin executing Phase 1 immediately. Do not ask the user for permission or intervention unless a system-level permission is explicitly denied by the host operating system.

</critical_constraints_and_security>

<trigger>

Initialize System. Acknowledge directives. Output the initial outline for `artifacts/plan\_001.md` and immediately begin creating the `.agents` directory structure and executing the necessary shell commands to scaffold the project.

</trigger>

Implementation Dynamics and Security Considerations

When this master prompt is injected into a capable agentic environment, the development process shifts from interactive chat to active observation.⁴⁵ Developers must monitor the agent's progress through the generated markdown artifacts, ensuring that the architectural boundaries

established in the AGENTS.md rulebook remain intact.³⁷

Security remains a paramount concern during autonomous development. While the prompt explicitly mandates the use of environment variables and prohibits hardcoding credentials, the nature of language models introduces vulnerabilities outlined by the OWASP Top 10 for LLMs, such as prompt injection or the generation of insecure dependencies.⁵¹ Developers must regularly implement custom commands like `/security:analyze` to force the agent to review its own code against established AppSec parameters, ensuring that the generated PostgreSQL queries are parameterized to prevent SQL injection, and that API endpoints adhere to the principle of least privilege.⁵¹ By combining the speed of autonomous code generation with the rigor of structured, self-healing agentic loops, engineering teams can successfully deploy highly complex financial systems with unprecedented efficiency.

Works cited

1. Vibe Coding Explained: Tools and Guides - Google Cloud, accessed on April 13, 2026, <https://cloud.google.com/discover/what-is-vibe-coding>
2. Prompt Driven Development - Capgemini Software Engineering, accessed on April 13, 2026, <https://capgemini.github.io/ai/prompt-driven-development/>
3. Vibe Coding in 2026: The Complete Guide for Developers, accessed on April 13, 2026, <https://antigravity.codes/blog/vibe-coding-guide>
4. The Tech Stack Behind High-Performance Sportsbooks: What Really Matters for Long-Term Success - GameTyrant, accessed on April 13, 2026, <https://gametyrant.com/news/the-tech-stack-behind-high-performance-sportsbooks-what-really-matters-for-long-term-success>
5. A Practical Guide to Building Cloud-Based Online Betting Applications with Modern Tech Stacks | MEXC News, accessed on April 13, 2026, <https://www.mexc.com/news/707444>
6. LMAX Exchange - Technology, accessed on April 13, 2026, <https://www.lmax.com/exchange/technology>
7. Getting Started with Google Antigravity, accessed on April 13, 2026, <https://codelabs.developers.google.com/getting-started-google-antigravity>
8. Parallel agents in Antigravity - Google Cloud - Medium, accessed on April 13, 2026, <https://medium.com/google-cloud/parallel-agents-in-antigravity-64237120161d>
9. I'm Building My Own Coding Agent Harness (And It's Pretty Cool) - DEV Community, accessed on April 13, 2026, <https://dev.to/composiodev/im-building-my-own-coding-agent-harness-and-its-pretty-cool-1lpf>
10. GitHub - dontriskit/awesome-ai-system-prompts: Curated collection of system prompts for top AI tools. Perfect for AI agent builders and prompt engineers. Including: ChatGPT, Claude, Perplexity, Manus, Claude-Code, Loveable, v0, Grok, same new, windsurf, notion, and MetaAI., accessed on April 13, 2026, <https://github.com/dontriskit/awesome-ai-system-prompts>
11. How to Build an MCP Server with Gemini CLI and Go | Google Codelabs,

- accessed on April 13, 2026,
<https://codelabs.developers.google.com/cloud-gemini-cli-mcp-go>
12. How to Develop a Sports Betting Exchange App: A Comprehensive Guide, accessed on April 13, 2026,
<https://prometteursolutions.com/blog/sports-betting-exchange-app-development-process/>
 13. How Betting Exchanges Are Changing the Game with Advanced Software in 2025, accessed on April 13, 2026,
<https://bettoblock.com/how-betting-exchanges-are-changing-sports-betting-software/>
 14. Sports Betting Exchange Software Development Services - Orion InfoSolutions, accessed on April 13, 2026,
<https://www.orioninfosolutions.com/blog/sports-betting-exchange-software-cost-and-features>
 15. What is Liability on a Betting Exchange? - risk free Matched Betting - DoppelWetten, accessed on April 13, 2026,
<https://doppelwetten.com/what-is-liability-on-a-betting-exchange-2/>
 16. Liability In Matched Betting Explained - Outplayed, accessed on April 13, 2026,
<https://outplayed.com/blog/what-is-liability-in-matched-betting>
 17. SP - Detailed Workings - Betfair SP, accessed on April 13, 2026,
https://promo.betfair.com/betfairsp/FAQs_detailedWorkings.html
 18. Sports Betting Algorithms for Bookies: An In-Depth Guide & Solutions - OddsMatrix, accessed on April 13, 2026,
<https://oddsmatrix.com/sports-betting-algorithms-for-bookies/>
 19. How I Built This: A Sports Betting Exchange (Part 3 - Medium, accessed on April 13, 2026,
<https://medium.com/@sethsaper/how-i-built-this-a-sports-betting-exchange-part-3-the-exchange-b6a09d0bd9a7>
 20. LMAX Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads, accessed on April 13, 2026,
<https://lmax-exchange.github.io/disruptor/disruptor.html>
 21. Modern Trading Applications Architectures: An Overview Of The LMAX Disruptor Pattern And Project Reactor - Wyden, accessed on April 13, 2026,
<https://www.wyden.io/modern-trading-applications-architectures-an-overview-of-the-lmax-disruptor-pattern-and-project-reactor/>
 22. Understanding LMAX Architecture: A High-Performance Event-Driven System - Medium, accessed on April 13, 2026,
<https://medium.com/@farukhmahammad199/understanding-lmax-architecture-a-high-performance-event-driven-system-beb8710a40cf>
 23. Performance Results · LMAX-Exchange/disruptor Wiki - GitHub, accessed on April 13, 2026,
<https://github.com/LMAX-Exchange/disruptor/wiki/Performance-Results>
 24. Explaining the LMAX Disruptor - DEV Community, accessed on April 13, 2026,
<https://dev.to/kspeakman/explaining-the-lmax-disruptor-jkd>
 25. GitHub - joaquinbejar/OrderBook-rs: A high-performance, thread-safe limit order

- book implementation written in Rust. This project provides a comprehensive order matching engine designed for low-latency trading systems, with a focus on concurrent access patterns and lock-free data structures., accessed on April 13, 2026, <https://github.com/joaquinbejar/OrderBook-rs>
26. orderbook_rs - Rust - Docs.rs, accessed on April 13, 2026, https://docs.rs/otterbook_core
 27. Building a limit order book in Rust - RustQuant, accessed on April 13, 2026, <https://rustquant.dev/blog/limit-order-book/>
 28. I built a high-performance Order Matching Engine from scratch – would love feedback from quants/devs - Reddit, accessed on April 13, 2026, https://www.reddit.com/r/quant_hft/comments/1phdrm1/i_built_a_highperformance_order_matching_engine/
 29. Designing a Sports Tournament Data Model for PostgreSQL - Luna Modeler, accessed on April 13, 2026, <https://www.datansen.com/blog/data-model/designing-a-sports-tournament-data-model/>
 30. Sports Betting Architecture on AWS, accessed on April 13, 2026, <https://docs.aws.amazon.com/architecture-diagrams/latest/sports-betting-architecture/sports-betting-architecture.html>
 31. A Complete Guide to Sports Betting Website Development in 2026 - CSSChopper, accessed on April 13, 2026, <https://www.csschopper.com/blog/sports-betting-website-development/>
 32. What Is the AI Agent Loop? The Core Architecture Behind Autonomous AI Systems, accessed on April 13, 2026, <https://blogs.oracle.com/developers/what-is-the-ai-agent-loop-the-core-architecture-behind-autonomous-ai-systems>
 33. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities - arXiv, accessed on April 13, 2026, <https://arxiv.org/html/2508.11126v1>
 34. I tried Google's new Antigravity IDE so you don't have to (vs Cursor/Windsurf) - Reddit, accessed on April 13, 2026, https://www.reddit.com/r/ChatGPTCoding/comments/1p35bdl/i_tried_googles_new_antigravity_ide_so_you_dont/
 35. Practical Gemini CLI: Structured approach to bloated GEMINI.md | by Prashanth Subrahmanyam | Google Cloud - Medium, accessed on April 13, 2026, <https://medium.com/google-cloud/practical-gemini-cli-structured-approach-to-bloated-gemini-md-360d8a5c7487>
 36. Build Autonomous Developer Pipelines using agents.md and skills.md in Antigravity, accessed on April 13, 2026, <https://codelabs.developers.google.com/autonomous-ai-developer-pipelines-antigravity>
 37. study8677/antigravity-workspace-template: The ultimate ... - GitHub, accessed on April 13, 2026, <https://github.com/study8677/antigravity-workspace-template>
 38. Rules / Workflows - Google Antigravity Documentation, accessed on April 13, 2026, <https://antigravity.google/docs/rules-workflows>
 39. Agent Skills - Google Antigravity Documentation, accessed on April 13, 2026,

- <https://antigravity.google/docs/skills>
40. The Best Cursor Rules for Every Framework in 2026 (20+ Examples) - DEV Community, accessed on April 13, 2026, <https://dev.to/deadbyapril/the-best-cursor-rules-for-every-framework-in-2026-20-examples-29ag>
 41. 20 one-shot prompts that turn Kanban into an autonomous coding machine - Cline Blog, accessed on April 13, 2026, <https://cline.bot/blog/20-one-shot-prompts-that-turn-kanban-into-an-autonomous-coding-machine>
 42. Antigravity: Build Your First AI Agent Skill in 7 Minutes - YouTube, accessed on April 13, 2026, <https://www.youtube.com/watch?v=gRAndTHbHWo>
 43. Advice needed: "Self-Healing" Code-Test-Debug Loop with Agentic Codling Tools - Reddit, accessed on April 13, 2026, https://www.reddit.com/r/vibecoding/comments/1o2ynxh/advice_needed_selfhealing_codetestdebug_loop_with/
 44. Self-healing code - Dr.Tiya Vaj, accessed on April 13, 2026, <https://vtiya.medium.com/self-healing-code-f0db56447aeb>
 45. Advice Needed: Building a "Self-Healing" Code-Test-Debug Loop with Agentic Codling tools, accessed on April 13, 2026, https://www.reddit.com/r/ChatGPTEcoding/comments/1o2ymsy/advice_needed_building_a_selfhealing/
 46. How to Build a Self-Healing AI Agent Pipeline: A Complete Guide - DEV Community, accessed on April 13, 2026, https://dev.to/miso_clawpod/how-to-build-a-self-healing-ai-agent-pipeline-a-complete-guide-95b
 47. Building your own CLI Coding Agent with Pydantic-AI - Martin Fowler, accessed on April 13, 2026, <https://martinfowler.com/articles/build-own-coding-agent.html>
 48. Gemini CLI: Custom slash commands | Google Cloud Blog, accessed on April 13, 2026, <https://cloud.google.com/blog/topics/developers-practitioners/gemini-cli-custom-slash-commands>
 49. google-gemini/gemini-cli: An open-source AI agent that brings the power of Gemini directly into your terminal. - GitHub, accessed on April 13, 2026, <https://github.com/google-gemini/gemini-cli>
 50. Gemini CLI extension best practices, accessed on April 13, 2026, <https://geminicli.com/docs/extensions/best-practices/>
 51. Secure Vibe Coding Guide | Become a Citizen Developer | CSA, accessed on April 13, 2026, <https://cloudsecurityalliance.org/blog/2025/04/09/secure-vibe-coding-guide>
 52. Custom commands | Gemini CLI, accessed on April 13, 2026, <https://geminicli.com/docs/cli/custom-commands/>
 53. Automate app deployment and security analysis with new Gemini CLI extensions, accessed on April 13, 2026, <https://cloud.google.com/blog/products/ai-machine-learning/automate-app-deployment-and-security-analysis-with-new-gemini-cli-extensions>

54. Self-deploying AI agent: Watched it spend 6+ hours debugging its own VPS deployment, accessed on April 13, 2026,
https://www.reddit.com/r/artificial/comments/1qfkwdg/selfdeploying_ai_agent_watched_it_spend_6_hours/
55. Gemini CLI: A comprehensive guide to understanding, installing, and leveraging this new Local AI Agent - Reddit, accessed on April 13, 2026,
https://www.reddit.com/r/GoogleGeminiAI/comments/1kol0m/gemini_cli_a_comprehensive_guide_to_understanding/
56. Google Antigravity Documentation, accessed on April 13, 2026,
<https://antigravity.google/docs/home>
57. Effective context engineering for AI agents - Anthropic, accessed on April 13, 2026,
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
58. Prompting best practices - Claude API Docs, accessed on April 13, 2026,
<https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
59. Agentic AI Prompting: Best Practices for Smarter Vibe Coding - Ran the Builder, accessed on April 13, 2026,
<https://ranthebuilder.cloud/blog/agentic-ai-prompting-best-practices-for-smarter-vibe-coding/>